

DOCUMENTACIÓN DE USO (USAGE_DOCS.md)

pygrep - Guía de Uso y Ejemplos

Tabla de Contenidos

1. [Instalación](#)
 2. [Uso Básico](#)
 3. [Referencia de Opciones](#)
 4. [Patrones de Búsqueda](#)
 5. [Contexto y Formato](#)
 6. [Manejo de Archivos](#)
 7. [Ejemplos del Mundo Real](#)
 8. [Solución de Problemas](#)
-

Instalación

Requisitos

- Python 3.10 o superior
- `charset-normalizer` (opcional, para `--auto-encoding`; se instala con `pip install charset-normalizer`)

Ejecución Directa

```
python pygrep.py [OPCIONES] PATRÓN [ARCHIVOS...]
```

Crear Ejecutable (Windows)

```
# Instalar PyInstaller
pip install pyinstaller

# Compilar
pyinstaller --onefile --name pynew pygrep.py

# Ejecutable generado en: dist/pynew.exe
.\dist\pynew.exe --help
```

Crear Ejecutable (Linux/macOS)

```
pyinstaller --onefile --name pynew pygrep.py
chmod +x dist/pynew
./dist/pynew --help
```

Uso Básico

Sintaxis General

```
pygrep [OPCIONES] PATRÓN [ARCHIVO_O_DIRECTORIO...]
```

Ejemplos Rápidos

Buscar en un archivo

```
pygrep "error" app.log
```

Buscar en múltiples archivos

```
pygrep "warning" file1.log file2.log file3.log
```

Buscar en todos los archivos de un tipo

```
pygrep "TODO" *.py  
pygrep "FIXME" *.js *.ts
```

Buscar recursivamente en un directorio

```
pygrep -r "function" src/
```

Leer desde stdin (pipe)

```
cat archivo.log | pygrep "ERROR"  
ps aux | pygrep "python"
```

Referencia Rápida de Opciones

Opción	Descripción	Ejemplo
--------	-------------	---------

Opción	Descripción	Ejemplo
<code>-V, --version</code>	Muestra la versión y sale	<code>grep_new -V</code>
<code>-, --help</code>	Muestra la ayuda y sale	<code>grep_new -?</code>
<code>-i</code>	Ignora mayúsculas/minúsculas	<code>grep_new -i "error" log.txt</code>
<code>-S, --smart-case</code>	Búsqueda inteligente: insensible si el patrón es todo minúsculas; sensible si contiene alguna mayúscula	<code>grep_new -S "error" log.txt</code>
<code>-v</code>	Invierte la coincidencia (líneas que NO coinciden)	<code>grep_new -v "DEBUG" log.txt</code>
<code>-w</code>	Coincide solo con palabras completas	<code>grep_new -w "def" code.py</code>
<code>-F</code>	Trata el patrón como texto literal (sin regex)	<code>grep_new -F ".log" files.txt</code>
<code>-f ARCHIVO</code>	Lee patrones desde un archivo (uno por línea)	<code>grep_new -f patterns.txt log.txt</code>
<code>-n</code>	Muestra el número de línea	<code>grep_new -n "ERROR" app.log</code>
<code>-c</code>	Solo imprime el conteo de coincidencias	<code>grep_new -c "warning" *.log</code>
<code>-m N</code>	Detiene la lectura tras N coincidencias	<code>grep_new -m 10 "error" big.log</code>
<code>-A N</code>	Muestra N líneas después del match	<code>grep_new -A 2 "Exception" app.log</code>
<code>-B N</code>	Muestra N líneas antes del match	<code>grep_new -B 1 "Error" app.log</code>
<code>-C N</code>	Muestra N líneas antes y después	<code>grep_new -C 3 "WARN" app.log</code>
<code>-r</code>	Búsqueda recursiva profunda en directorios	<code>grep_new -r "TODO" src/</code>
<code>-l</code>	Solo imprime nombres de archivos con coincidencia	<code>grep_new -l "ERROR" *.log</code>
<code>-L</code>	Solo imprime nombres de archivos sin coincidencia	<code>grep_new -L "DEBUG" *.log</code>
<code>-h</code>	Suprime el nombre del archivo en la salida	<code>grep_new -h "error" a.log b.log</code>
<code>-H</code>	Fuerza la impresión del nombre del archivo	<code>grep_new -H "error" solo.log</code>
<code>--color WHEN</code>	<code>auto, always, never</code>	<code>grep_new --color=always "x" f.txt</code>
<code>--encoding ENC</code>	Codificación del archivo (default: <code>utf-8</code>)	<code>grep_new --encoding latin-1 "café" f.txt</code>

Opción	Descripción	Ejemplo
<code>--auto-encoding</code>	Detecta automáticamente el encoding de cada fichero (usa <code>charset-normalizer</code>). Avisa por <code>stderr</code> si difiere del configurado	<code>grep_new --auto-encoding "patrón" fichero.txt</code>
<code>--redos-strict</code>	Rechaza patrones con riesgo de ReDoS	<code>grep_new --redos-strict "(a+)+" f.txt</code>
<code>--stats</code>	Muestra estadísticas de búsqueda al finalizar	<code>grep_new "error" app.log --stats</code>
<code>-u, --unique</code>	Elimina coincidencias duplicadas	<code>grep_new "error" app.log --unique</code>
<code>--include PATRÓN</code>	Incluye solo archivos que coincidan	<code>grep_new "error" *.log --include="*.log"</code>
<code>--exclude PATRÓN</code>	Excluye archivos que coincidan	<code>grep_new "error" *.log --exclude="*.bak"</code>
<code>--ignore-dir DIR</code>	Ignora directorios con este nombre	<code>grep_new "error" src/ --ignore-dir="node_modules"</code>
<code>--output FICHERO</code>	Guarda resultados en un archivo	<code>grep_new "error" app.log --output=resultados.txt</code>
<code>--format FORMAT</code>	Formato de salida: <code>text</code> , <code>json</code> , <code>csv</code>	<code>grep_new "error" app.log --output=resultados.csv --format=csv</code>
<code>-z, --search-zip</code>	Buscar en archivos comprimidos (<code>.gz</code> , <code>.bz2</code> , <code>.xz/.lzma</code> , <code>.zip</code> , <code>.tar</code> , <code>.tar.gz</code> , <code>.tgz</code> , <code>.tar.bz2</code> , <code>.tar.xz</code>)	<code>grep_new -z "error" logs.zip</code>
<code>-o, --extract</code>	Muestra solo las partes de la línea que coinciden con el patrón	<code>grep_new -o "[0-9]+" app.log</code>
<code>--summary</code>	Muestra un resumen corto de una línea de la búsqueda al finalizar	<code>grep_new "error" app.log --summary</code>
<code>-U, --multiline</code>	Permitir coincidencias de patrones que abarquen múltiples líneas	<code>grep_new -U "hello\nworld" file.txt</code>
<code>--tail N</code>	Muestra solo las últimas N coincidencias del patrón	<code>grep_new "ERROR" app.log --tail 20</code>
<code>--time-range INICIO FIN</code>	Filtra coincidencias por rango de timestamp	<code>grep_new "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 12:00"</code>
<code>--follow</code>	Seguir el archivo en tiempo real (similar a <code>tail -f</code>)	<code>grep_new "ERROR" app.log --follow</code>

Patrones de Búsqueda

1. Patrones Básicos

Texto simple

```
# Buscar la palabra "error"
pygrep "error" app.log

# Buscar "Error" con mayúscula exacta
pygrep "Error" app.log

# Ignorar mayúsculas/minúsculas
pygrep -i "error" app.log
# Matchea: error, Error, ERROR, eRrOr, etc.

# Smart-case: insensible porque el patrón es todo minúsculas
pygrep -S "error" app.log
# Matchea: error, Error, ERROR ...

# Smart-case: sensible porque el patrón contiene mayúscula
pygrep -S "Error" app.log
# Solo matchea: Error

# Errores únicos (sin repeticiones)
pygrep "Exception" app.log --unique
```

Múltiples palabras (OR lógico)

```
# Buscar "error" O "warning" O "critical"
pygrep "error|warning|critical" app.log

# Con ignore-case
pygrep -i "error|warn|fatal" app.log
```

2. Expresiones Regulares (Regex)

Caracteres especiales

```
# Punto (.) - cualquier caracter
pygrep "err.r" app.log
# Matchea: error, errar, err9r, etc.

# Asterisco (*) - cero o más repeticiones
pygrep "colou*r" app.log
# Matchea: color, colour
```

```
# Más (+) - una o más repeticiones
pygrep "go+gle" app.log
# Matchea: google, gooogle, goooogle

# Interrogación (?) - cero o una vez
pygrep "https?" app.log
# Matchea: http, https

# Corchetes ([]) - clase de caracteres
pygrep "[aeiou]" app.log
# Matchea cualquier vocal

pygrep "[0-9]" app.log
# Matchea cualquier dígito

pygrep "[a-zA-Z]" app.log
# Matchea cualquier letra

# Corchetes negados ([^])
pygrep "[^0-9]" app.log
# Matchea cualquier cosa que NO sea dígito
```

Anclajes

```
# Inicio de línea (^)
pygrep "^ERROR" app.log
# Solo matchea si ERROR está al inicio

# Fin de línea ($)
pygrep "failed$" app.log
# Solo matchea si failed está al final

# Palabra completa (^ y $ combinados)
pygrep "^INFO$" app.log
# Solo matchea líneas que son exactamente "INFO"
```

Cuantificadores

```
# Exactamente N veces {n}
pygrep "[0-9]{4}" app.log
# Matchea 4 dígitos: 2024, 1999, etc.

# De N a M veces {n,m}
pygrep "[0-9]{2,4}" app.log
# Matchea de 2 a 4 dígitos: 24, 123, 2024

# Al menos N veces {n,}
```

```
pygrep "a{3,}" app.log
# Matchea: aaa, aaaa, aaaaa, etc.
```

Grupos y Capturas

```
# Grupo con paréntesis
pygrep "(error|warn)ing" app.log
# Matchea: erroring, warning

# Grupo sin captura (?)
pygrep "(?:https?|ftp)://" app.log
# Matchea: http://, https://, ftp://

# Backreference (\1, \2, etc.)
pygrep "([a-z])\1" app.log
# Matchea letras repetidas: ll, ee, tt, etc.
```

3. Patrones con -F (Fixed Strings)

Cuando usas -F, el patrón se trata como texto literal, NO como regex:

```
# Buscar ".log" literalmente (el punto NO es wildcard)
pygrep -F ".log" files.txt

# Buscar caracteres especiales sin escapar
pygrep -F "error: [timeout]" app.log
# Busca literalmente: error: [timeout]

# Sin -F, tendrías que escapar:
pygrep "error: \[timeout\]" app.log
```

4. Patrones con -w (Word Boundary)

```
# Buscar "def" como palabra completa
pygrep -w "def" code.py
# Matchea: def foo():
# NO matchea: default, subclass, define

# Combinar -F y -w (correctamente implementado)
pygrep -F -w ".env" config.txt
# Matchea: .env
# NO matchea: .env.backup, my.env
```

5. Búsqueda Inteligente de Mayúsculas (-S / --smart-case)

Con **-S**, no tienes que decidir tú si necesitas **-i** o no: **el propio patrón decide**.

Patrón	Comportamiento
Todo en minúsculas	Actúa como -i (ignora mayúsculas/minúsculas)
Contiene alguna mayúscula	Búsqueda estricta (sensible a mayúsculas)

Prioridad: Si se combinan **-i** y **-S**, **-i** gana siempre (siempre insensible).

```
# Patrón todo minúsculas → insensible (busca error, Error, ERROR...)
pygrep -S "error" app.log

# Patrón con mayúscula → sensible (solo busca Error)
pygrep -S "Error" app.log

# Patrón con mayúsculas múltiples → sensible (solo busca CRITICAL)
pygrep -S "CRITICAL" app.log

# Útil con regex: patrón minúscula → insensible
pygrep -S "error|warning|fatal" app.log
# Matchea: ERROR, error, Warning, FATAL, etc.

# Regex con mayúscula → sensible (solo la forma exacta)
pygrep -S "Error|Warning" app.log

# Combinado con número de línea
pygrep -S -n "exception" app.log

# En búsqueda recursiva
pygrep -S -r "todo" src/
# Matchea TODO, todo, Todo... porque el patrón es todo minúsculas
```

6. Múltiples Patrones con **-f**

Crea un archivo **patrones.txt**:

```
error
warning
critical
fatal
```

Úsalo:

```
pygrep -f patrones.txt app.log

# Combinar con otras opciones
```

```
pygrep -f patrones.txt -i -n app.log
pygrep -f patrones.txt -C 3 src/
```

Combinar patrón posicional + archivo:

```
# Busca: "exception" O cualquier patrón en patrones.txt
pygrep -f patrones.txt "exception" app.log
```

6. Ejemplos de Patrones Complejos

Direcciones IP

```
# IPv4 simple
pygrep "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log

# IPv4 más preciso (0-255)
pygrep "([0-9]{1,3}\.){3}[0-9]{1,3}" access.log
```

Fechas

```
# Formato YYYY-MM-DD
pygrep "[0-9]{4}-[0-9]{2}-[0-9]{2}" logs.txt

# Formato DD/MM/YYYY
pygrep "[0-9]{2}/[0-9]{2}/[0-9]{4}" logs.txt

# Múltiples formatos
pygrep "[0-9]{4}-[0-9]{2}-[0-9]{2}|[0-9]{2}/[0-9]{2}/[0-9]{4}" logs.txt
```

Emails

```
pygrep "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" contacts.txt
```

URLs

```
pygrep "https?://[a-zA-Z0-9./_-]+" web.log
```

Códigos HTTP

```
# Códigos 4xx y 5xx
pygrep "HTTP/[0-9.]+ [45][0-9]{2}" access.log

# Solo errores 500
pygrep "HTTP/[0-9.]+ 500" access.log
```

Timestamps

```
# Formato ISO 8601
pygrep "[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}" app.log

# Formato común de logs
pygrep "[0-9]{2}:[0-9]{2}:[0-9]{2}" syslog.log
```

Números de teléfono

```
# Formato internacional
pygrep "\+[0-9]{1,3}[- ]?[0-9]{3,4}[- ]?[0-9]{3}[- ]?[0-9]{3,4}" contacts.txt
```

Código Python

```
# Definiciones de funciones
pygrep "^\\s*def [a-zA-Z_][a-zA-Z0-9_]*\\(" *.py

# Imports
pygrep "^from [a-zA-Z0-9_.]+ import" *.py

# Clases
pygrep "^class [A-Z][a-zA-Z0-9_]*" *.py

# Decoradores
pygrep "^\\s*@[a-zA-Z_][a-zA-Z0-9_]*" *.py
```

JSON

```
# Claves JSON
pygrep '"[a-zA-Z_][a-zA-Z0-9_]*\\s*:' data.json

# Valores string
pygrep '":\\s*"([^"])*"' data.json
```

7. Patrones Peligrosos (ReDoS)

Estos patrones pueden causar bloqueo por ReDoS:

```
# Quantifiers anidados - PELIGROSO
pygrep "(a+)+$" archivo.txt
pygrep "([a-z]+)+$" archivo.txt
pygrep "((a|aa)+)+$" archivo.txt

# Con --redos-strict, serán rechazados:
pygrep --redos-strict "(a+)+$" archivo.txt
# pygrep: patrón rechazado por --redos-strict: (a+)+$
```

Contexto y Formato

Mostrar Líneas de Contexto

-A N (After)

```
# Mostrar 3 líneas después de cada match
pygrep -A 3 "ERROR" app.log

# Ejemplo de salida:
# 45:ERROR: Connection failed
# 46-Retrying in 5 seconds...
# 47-Attempt 1 of 3
# 48-Waiting for response
```

-B N (Before)

```
# Mostrar 2 líneas antes de cada match
pygrep -B 2 "Exception" app.log

# Ejemplo de salida:
# 102-Processing request #456
# 103-Validating input
# 104:Exception: Invalid parameter
```

-C N (Context - antes y después)

```
# Mostrar 5 líneas antes y después
pygrep -C 5 "WARNING" app.log
```

```
# Ejemplo de salida:
# 200-Starting batch job
# 201-Loading configuration
# 202-Connecting to database
# 203:WARNING: Connection pool exhausted
# 204-Creating new connections
# 205-Processing records
# 206-Batch complete
```

Grupos Solapados

Cuando dos matches están cerca, sus contextos se fusionan:

```
# Archivo log.txt:
# 1: ERROR en línea 1
# 2: INFO línea 2
# 3: ERROR en línea 3
# 4: INFO línea 4

pygrep -C 1 "ERROR" log.txt

# Salida (sin separador -- porque los contextos se solapan):
# 1:ERROR en línea 1
# 2-INFO línea 2
# 3:ERROR en línea 3
# 4-INFO línea 4
```

Si los grupos están separados:

```
pygrep -C 1 "línea 1|línea 4" log.txt

# Salida (con separador --):
# 1:ERROR en línea 1
# 2-INFO línea 2
# --
# 3-ERROR en línea 3
# 4:INFO línea 4
```

Números de Línea

```
pygrep -n "pattern" archivo.txt

# Salida:
# 45:match encontrado
# 128:otro match
```

Conteo de Coincidencias

```
pygrep -c "ERROR" app.log
# Salida: 23

# Con múltiples archivos:
pygrep -c "ERROR" *.log
# Salida:
# app.log:23
# error.log:156
# system.log:5
```

Colores

```
# Automático (solo en terminal)
pygrep --color=auto "ERROR" app.log

# Siempre (útil para pipes a less)
pygrep --color=always "ERROR" app.log | less -R

# Nunca
pygrep --color=never "ERROR" app.log
```

Estadísticas de Búsqueda

Mostrar estadísticas completas

```
pygrep "error" app.log --stats

# Salida (en stderr):
# [ Estadísticas de búsqueda ]
# -----
# Archivos analizados      : 1
# Archivos con matches     : 1
# Líneas procesadas       : 14
# Coincidencias totales    : 1
# Tasa de coincidencia     : 7.14%
# Tiempo transcurrido      : 0.001s
#
# Top archivos con mas coincidencias:
#   test.txt               1
# -----
```

Combinar con otras opciones

```
# Con ignore-case, line-number y unique
pygrep -i -n -u "error" test.txt --stats
# Muestra estadísticas AND resultados únicos
```

Resumen de Nivel de Log (--summary)

Muestra un resumen en formato tabla con el conteo y porcentaje de cada nivel de log (ERROR, WARN, INFO, DEBUG, FATAL, CRITICAL) encontrado en los resultados de la búsqueda:

```
pygrep . app.log --summary

# Salida:
# Nivel | Ourrencias | Porcentaje
# -----|-----|-----
# ERROR | 127         | 2.8%
# WARN  | 384         | 8.4%
# INFO  | 3,891       | 85.1%
# DEBUG | 167         | 3.7%
```

Conteo por archivo

```
pygrep -c "error" *.log
```

unique: Eliminar duplicados

```
# Sin --unique (muestra todas las coincidencias)
pygrep "error" log.txt

# Con --unique (solo una vez por tipo de match)
pygrep "error" log.txt --unique
# Solo muestra: ERROR: Connection failed
# Omite: ERROR: Connection failed (duplicado)
```

extract / only-matching: Extraer coincidencia

La opción `-o`, `--extract` o `--only-matching` extrae y muestra únicamente la parte de la línea que coincide con el patrón de búsqueda (en lugar de imprimir la línea completa).

Si una misma línea contiene varias coincidencias, cada coincidencia se muestra en una línea nueva.

```
# Mostrar solo las direcciones IP de un log
pygrep -o "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log

# Con número de línea
pygrep -o -n "[0-9]+" log.txt
# Muestra:
# 1:123
# 1:456
# 3:789
```

multiline: Búsqueda Multilínea

La opción `-U` o `--multiline` permite que los patrones de búsqueda coincidan a lo largo de varias líneas. Cuando esta opción está activa, la búsqueda se realiza sobre el archivo completo como una única cadena en lugar de procesar el archivo línea por línea.

Cada línea de texto dentro del bloque coincidente se imprime de forma individual conservando su número de línea original correspondiente.

```
# Buscar un bloque de texto que abarque un salto de línea
pygrep -U "hello\nworld" test.txt

# Buscar bloques de clase de Python
pygrep -U "class [\s\S]*?:" src/ -r -n
```

Últimas N Coincidencias (`--tail`)

La opción `--tail N` muestra únicamente las **últimas N coincidencias** del patrón, similar a cómo `tail` muestra las últimas líneas de un archivo pero aplicado a los resultados de búsqueda.

```
# Las 10 últimas ocurrencias de ERROR en el log
pygrep "ERROR" app.log --tail 10

# Las 5 últimas con número de línea
pygrep "ERROR" app.log --tail 5 -n

# Las 3 últimas con contexto de 2 líneas antes y después
pygrep "CRITICAL" app.log --tail 3 -C 2

# Las 20 últimas en todos los logs del directorio
pygrep -r "Exception" logs/ --tail 20

# Combinado con --time-range
pygrep "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 12:00" --tail 5
```

Nota: `--tail` opera sobre las coincidencias del patrón, no sobre las líneas del archivo. Si se usa con contexto (`-A`, `-B`, `-C`), las líneas de contexto de cada coincidencia se preservan, por lo que la salida puede tener más de N líneas en total.

Tail 0: `--tail 0` no muestra ninguna coincidencia (salida vacía).

Filtrado por Rango de Tiempo (`--time-range`)

La opción `--time-range` **INICIO** **FIN** filtra las coincidencias mostrando solo aquellas cuya línea contenga un timestamp dentro del intervalo especificado.

Formatos de timestamp soportados

Formato	Ejemplo	Descripción
YYYY-MM-DDTHH:MM:SS	2026-05-26T10:30:00	ISO 8601 con T
YYYY-MM-DD HH:MM:SS	2026-05-26 10:30:00	ISO 8601 con espacio
YYYY-MM-DD HH:MM	2026-05-26 10:30	ISO sin segundos
DD/Mon/YYYY:HH:MM:SS	26/May/2026:10:30:00	Formato Apache access log
Mon DD HH:MM:SS	May 26 10:30:00	Formato syslog

Ejemplos

```
# Filtrar logs de una hora concreta (ISO con espacio)
pygrep "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 11:00"

# Con formato ISO 8601 completo
pygrep "Exception" app.log --time-range "2026-05-26T08:00:00" "2026-05-26T09:30:00"

# Logs de Apache: buscar errores en un rango
pygrep "500" access.log --time-range "26/May/2026:10:00:00" "26/May/2026:11:00:00"

# Formato syslog
pygrep "error" /var/log/syslog --time-range "May 26 10:00:00" "May 26 12:00:00"

# Combinado con --tail: las últimas 5 coincidencias de ERROR en una hora
pygrep "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 11:00" --tail 5

# Con número de línea y contexto
pygrep "CRITICAL" app.log --time-range "2026-05-26 10:00" "2026-05-26 12:00" -n -C 1

# En múltiples archivos con wildcards
pygrep "ERROR" "*.log" --time-range "2026-05-26 00:00" "2026-05-26 06:00"
```

Líneas sin timestamp

Las líneas que **no contienen ningún timestamp reconocible** se excluyen cuando `--time-range` está activo.

Validaciones

```
# Error: timestamp de inicio inválido
pygrep "ERROR" app.log --time-range "not-a-date" "2026-05-26 12:00"
# pygrep: --time-range: timestamp de inicio inválido: 'not-a-date'

# Error: inicio posterior al fin
pygrep "ERROR" app.log --time-range "2026-05-26 12:00" "2026-05-26 10:00"
# pygrep: --time-range: el inicio debe ser anterior al fin
```

Seguir Archivos en Tiempo Real (`--follow`)

La opción `--follow` permite monitorear un archivo en tiempo real a medida que se le agregan nuevos datos, de manera idéntica al comportamiento de `tail -f`.

Al activar esta opción:

1. El programa se posiciona directamente al final del archivo especificado.
2. Espera a que se agreguen nuevas líneas de texto y filtra en tiempo real las coincidencias basadas en el patrón.
3. Se mantiene ejecutando indefinidamente hasta que sea interrumpido (usualmente mediante `Ctrl+C`).

Ejemplos de uso

```
# Seguir un log imprimiendo en tiempo real solo las líneas que contengan "ERROR"
pygrep "ERROR" app.log --follow

# Seguir el archivo mostrando el número de línea real e ignorando
mayúsculas/minúsculas
pygrep -i -n "critical" production.log --follow

# Resaltar en tiempo real un término específico con color
pygrep "warning" system.log --follow --color=always
```

Limitaciones y Restricciones

Debido a su naturaleza en tiempo real e interactiva, `--follow` no es compatible con ciertas opciones de filtrado diferido o estadísticas. Si intentas usarlas juntas, `pygrep` mostrará un error y saldrá de inmediato:

- **Incompatible con:** `-c` / `--count`, `--stats`, `--summary`, `--tail`, `--time-range`, `--output`, `-r` / `--recursive`, `-l` / `--files-with-matches` y `-L` / `--files-without-match`.
- Requiere especificar al menos un archivo regular en los argumentos (no funciona directamente con la entrada estándar `stdin` redireccionada `-`).

📁 Control de Nombres de Archivos

Combinación	Comportamiento
Sin <code>-h</code> ni <code>-H</code>	Automático: muestra nombre si hay >1 archivo, comodines o <code>-r</code>
<code>-H</code>	Siempre muestra <code>nombre_archivo:</code>
<code>-h</code>	Nunca muestra nombre (prioridad máxima)
<code>-l</code>	Imprime solo el nombre del archivo (1 vez por archivo)
<code>-L</code>	Imprime solo el nombre del archivo si no hay coincidencias

Ejemplos:

```
# Forzar nombre incluso con 1 archivo
grep_new -H -n "error" test.txt
# Salida: test.txt:45:# 45:ERROR: Connection failed

# Suprimir nombres con múltiples archivos
grep_new -h -i "warn" a.log b.log c.log
# Salida: 12:warning... (sin prefijo)
```

Manejo de Archivos

Archivos Individuales

```
pygrep "pattern" archivo.txt
pygrep "pattern" /ruta/completa/archivo.log
```

Múltiples Archivos

```
pygrep "pattern" file1.txt file2.txt file3.txt
```

Comodines (Wildcards)

Importante en Windows: Siempre usa comillas dobles alrededor del patrón.

```
# Todos los archivos .log
pygrep "ERROR" "*.log"

# Todos los archivos .txt
pygrep "TODO" "*.txt"
```

```
# Archivos que empiezan con "app"
pygrep "warning" "app*.log"

# Archivos que terminan con número
pygrep "error" "*.log.1"

# Múltiples extensiones
pygrep "import" "*.py" "*.js" "*.ts"
```

Búsqueda Recursiva

```
# Sin -r: solo archivos directos del directorio
pygrep "def" src/
# Busca en: src/main.py, src/utils.py
# NO busca en: src/subdir/module.py

# Con -r: recursivo completo
pygrep -r "def" src/
# Busca en: src/main.py, src/utils.py, src/subdir/module.py, etc.
```

Directorios Específicos

```
# Múltiples directorios
pygrep -r "TODO" src/ tests/ docs/

# Mezclar archivos y directorios
pygrep "ERROR" app.log src/ backup/
```

```
# ignore-dir: Ignorar directorios
pygrep "error" src/ --ignore-dir=".git" --ignore-dir="node_modules" --ignore-dir="vendor"
```

```
# include: Incluir solo archivos que coincidan
pygrep "error" *.log --include="*.log"
```

```
# exclude: Excluir archivos que coincidan
pygrep "error" *.log --exclude="*.bak"
```

Archivos Ocultos

Por defecto, **sí** se buscan archivos ocultos (que empiezan con `.`):

```
pygrep -r "SECRET" .  
# Busca en: .env, .git/config, .hidden_file, etc.
```

Archivos Binarios

```
# Si detecta un archivo binario:  
pygrep "ERROR" binary.dat  
# Salida: El archivo binario binary.dat coincide  
  
# Con -c:  
pygrep -c "ERROR" binary.dat  
# Salida: binary.dat:0 (o 1 si coincide)
```

Archivos Comprimidos

Usa `-z` o `--search-zip` para buscar dentro de archivos comprimidos. Se admiten los siguientes formatos:

Extensión	Tipo
<code>.gz</code>	gzip individual
<code>.bz2</code>	bzip2 individual
<code>.xz</code> , <code>.lzma</code>	xz/lzma individual
<code>.zip</code>	ZIP (múltiples ficheros internos)
<code>.tar</code>	TAR sin comprimir (múltiples ficheros internos)
<code>.tar.gz</code> , <code>.tgz</code>	TAR comprimido con gzip
<code>.tar.bz2</code> , <code>.tbz2</code>	TAR comprimido con bzip2
<code>.tar.xz</code> , <code>.txz</code>	TAR comprimido con xz
<code>.tar.lzma</code>	TAR comprimido con lzma

```
# Buscar en archivos gzip individuales  
pygrep -z "patrón" archivo.gz  
  
# Buscar en un archivo zip (busca en todos los ficheros del zip)  
pygrep -z "error" logs.zip  
  
# Buscar en un tar sin comprimir  
pygrep -z "error" backup.tar  
  
# Buscar en un tar.gz / tgz  
pygrep -z "Exception" backup.tar.gz  
pygrep -z "Exception" backup.tgz
```

```
# Buscar en tar.bz2
pygrep -z "error" logs.tar.bz2

# Buscar en tar.xz
pygrep -z "error" logs.tar.xz

# Con número de línea y estadísticas
pygrep -z -n "Exception" logs.tar.gz --stats

# Solo listar ficheros internos que coincidan
pygrep -z -l "error" backup.tar.gz

# Filtrar ficheros internos por extensión (con --include)
pygrep -z --include="*.log" "ERROR" backup.tar.gz
```

Nota: Los ficheros que hay dentro del TAR/ZIP se muestran con la ruta virtual

`archivo.tar:nombre_interno`. Por ejemplo: `backup.tar.gz:var/log/app.log:2:error crítico`

Codificación de Archivos

```
# UTF-8 (default)
pygrep "café" archivo.txt

# Latin-1 (ISO-8859-1)
pygrep --encoding latin-1 "café" archivo_latin1.txt

# Windows-1252
pygrep --encoding cp1252 "café" archivo_windows.txt

# UTF-16
pygrep --encoding utf-16 "café" archivo_utf16.txt
```

Detección automática de encoding (`--auto-encoding`)

Cuando no conoces el encoding del fichero, usa `--auto-encoding`. pygrep leerá los primeros bytes del fichero y usará `charset-normalizer` para detectar el encoding real antes de procesarlo:

```
# Dejar que pygrep detecte el encoding automáticamente
pygrep --auto-encoding "patrón" fichero_desconocido.txt

# Si el encoding detectado difiere del configurado, se avisa por stderr:
# pygrep: fichero_desconocido.txt: encoding detectado 'windows-1252' (configurado: 'utf-8')

# Combinado con --encoding como fallback explícito
pygrep --auto-encoding --encoding utf-8 "patrón" fichero.txt

# Útil en búsquedas recursivas con ficheros de distintos orígenes
pygrep -r --auto-encoding "patrón" directorio/
```

Nota: `--auto-encoding` opera sobre los primeros 8 192 bytes del fichero, los mismos que se usan para detectar si es binario, sin coste adicional de I/O. Si `charset-normalizer` no está instalado, cae silenciosamente al encoding configurado (`--encoding`, por defecto `utf-8`).

```
# Guardar resultados en un archivo
pygrep "error" app.log --output=resultados.txt
```

```
# Guardar en JSON
pygrep "error" app.log --output=resultados.json --format=json
```

```
# Guardar en CSV
pygrep "error" app.log --output=resultados.csv --format=csv
```

stdin (Pipes)

```
# Pipe desde cat
cat archivo.log | pygrep "ERROR"

# Pipe desde otro comando
ps aux | pygrep "python"

# Pipe con head
pygrep -n "ERROR" bigfile.log | head -n 20

# Múltiples pipes
cat archivo.log | pygrep "ERROR" | pygrep -v "WARNING"
```

Ejemplos del Mundo Real

1. Debugging de Logs

```
# Encontrar todos los errores con contexto
pygrep -C 3 "ERROR" /var/log/app.log

# Errores críticos o fatales
pygrep -i "critical|fatal" /var/log/syslog

# Errores en un rango de tiempo
pygrep "2024-01-15 1[0-9]:[0-9]{2}:" app.log
```

```
# Stack traces de Python
pygrep -A 10 "Traceback" app.log

# Excepciones específicas
pygrep "Exception:|Error:" app.log
```

2. Análisis de Código

```
# Encontrar funciones no usadas (heurísticamente)
pygrep -r "^s*def [a-z_]+" src/ | pygrep -v "test_"

# Imports duplicados
pygrep "^import " *.py | sort | uniq -d

# TODOs y FIXMEs
pygrep -r -n "TODO|FIXME|XXX|HACK" src/

# Funciones largas (más de 50 líneas)
pygrep -A 50 "^s*def " src/*.py | pygrep -c "^s*def"

# Clases sin docstring
pygrep -A 3 "^class " src/*.py | pygrep -v '"""'
```

3. Seguridad

```
# Buscar contraseñas hardcodeadas
pygrep -r -i "password\s*=\s*['\"][^'\"]+['\"]" src/

# Buscar tokens de API
pygrep -r "api[_-]?key\s*[:=]\s*['\"][a-zA-Z0-9]{20,}" src/

# Buscar URLs con credenciales
pygrep "https?://[^:]+:[^@]+@" config/

# Buscar emails expuestos
pygrep -r "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" src/
```

4. DevOps y Sysadmin

```
# IPs que acceden al servidor
pygrep -o "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log | sort |
uniq -c | sort -rn

# Códigos HTTP 404
pygrep "HTTP/[0-9.]+ 404" access.log
```

```
# Requests lentos (más de 5 segundos)
pygrep "[0-9]{4,}" access.log # Asumiendo que el último número es tiempo en ms

# Errores de base de datos
pygrep -i "mysql|postgres|mongodb.*error" app.log

# Uso de memoria/CPU
pygrep "memory|cpu|ram" syslog.log
```

5. Desarrollo Web

```
# Endpoints de API
pygrep -r "@app.route\|@router\." src/

# Consultas SQL
pygrep -r "SELECT|INSERT|UPDATE|DELETE" src/

# Variables de entorno
pygrep -r "os.environ\|process.env" src/

# Promesas/async
pygrep -r "\.then\(\|async \|await " src/

# Console.logs de debug
pygrep -r "console.log\|print(" src/
```

6. Análisis de Datos

```
# Contar ocurrencias de una palabra
pygrep -o "python" archivo.txt | wc -l

# Extraer emails únicos
pygrep -o "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" contacts.txt | sort -u

# Extraer URLs
pygrep -o "https?://[a-zA-Z0-9./_-]+" web.log

# Extraer números de teléfono
pygrep -o "\+[0-9]{1,3}[- ]?[0-9]{10,}" contacts.txt
```

7. Git y Version Control

```
# Buscar en commits (con git log)
git log --all --full-history -- "*.py" | pygrep "TODO|FIXME"

# Buscar en diff
```

```
git diff | pygrep "^+.*TODO"

# Buscar en archivos ignorados
pygrep -r "SECRET" . --exclude-dir=.git
```

8. Testing

```
# Tests fallidos
pygrep "FAILED|ERROR" test_output.txt

# Coverage bajo
pygrep -B 2 "0%" coverage_report.txt

# Assertions fallidas
pygrep -A 5 "AssertionError" test.log
```

Solución de Problemas

1. Error: "down" no se reconoce como un comando

Problema: En Windows, el símbolo `|` es interpretado por la consola como pipe.

Solución: Usa comillas dobles alrededor del patrón:

```
REM ✗ Incorrecto:
grep_new.exe -n -i -w up|down archivo.txt

REM ☑ Correcto:
grep_new.exe -n -i -w "up|down" archivo.txt

REM Alternativa: Escapar con ^
grep_new.exe -n -i -w up^|down archivo.txt
```

2. Error: No existe el archivo o directorio

Causas posibles:

- El archivo no existe
- La ruta es incorrecta
- No tienes permisos de lectura

Solución:

```
# Verificar que el archivo existe
ls -la archivo.txt # Linux/macOS
dir archivo.txt    # Windows
```

```
# Usar ruta absoluta
pygrep "pattern" /ruta/completa/archivo.txt

# Verificar permisos
chmod +r archivo.txt # Linux/macOS
```

3. Error: Expresión regular inválida

Causa: El patrón regex tiene sintaxis incorrecta.

Ejemplos comunes:

```
# Paréntesis sin cerrar
pygrep "(error" archivo.txt # ✗

# Corchete sin cerrar
pygrep "[a-z" archivo.txt # ✗

# Backslash al final
pygrep "path\" archivo.txt # ✗
```

Solución: Escapar caracteres especiales o usar `-F`:

```
# Escapar
pygrep "\(error" archivo.txt

# O usar fixed strings
pygrep -F "(error" archivo.txt
```

4. Sin resultados (pero debería haber)

Verificaciones:

```
# 1. ¿El archivo existe y tiene contenido?
ls -la archivo.txt

# 2. ¿Estás buscando en el archivo correcto?
pygrep -n "pattern" archivo.txt

# 3. ¿Necesitas -i para ignore-case?
pygrep -i "ERROR" archivo.txt

# 4. ¿El patrón es correcto?
# Probar con un patrón simple
pygrep "test" archivo.txt
```

```
# 5. ¿Es un archivo binario?  
file archivo.txt # Linux/macOS
```

5. Resultados incorrectos con `-w`

Problema: `-w` no funciona como esperas con caracteres especiales.

Ejemplo:

```
# Buscar ".env" como palabra completa  
pygrep -w ".env" config.txt  
  
# Si no funciona, usar -F -w  
pygrep -F -w ".env" config.txt
```

6. Lento con archivos grandes

Soluciones:

```
# 1. Usar -m para limitar matches  
pygrep -m 100 "pattern" bigfile.log  
  
# 2. Usar head en el output  
pygrep "pattern" bigfile.log | head -n 100  
  
# 3. Filtrar por fecha primero  
pygrep "2024-01-15" bigfile.log | pygrep "ERROR"  
  
# 4. Usar grep nativo si está disponible (más rápido)  
grep "pattern" bigfile.log
```

7. Colores no funcionan en Windows

Solución:

```
# Forzar colores  
pygrep --color=always "pattern" archivo.txt | more  
  
# O usar Windows Terminal en lugar de cmd.exe  
# Windows Terminal soporta ANSI colors nativamente
```

8. Error de encoding

Problema: El fichero no es UTF-8 y aparece un error de decodificación o caracteres extraños.

Solución más sencilla — dejar que pygrep detecte el encoding automáticamente:

```
pygrep --auto-encoding "pattern" archivo.txt
# Si detecta, por ejemplo, windows-1252, avisa por stderr y busca correctamente
```

Solución manual — especificar el encoding conocido:

```
# Probar con latin-1
pygrep --encoding latin-1 "pattern" archivo.txt

# Probar con cp1252 (Windows)
pygrep --encoding cp1252 "pattern" archivo.txt

# UTF-16 (ficheros Windows con BOM)
pygrep --encoding utf-16 "pattern" archivo.txt
```

Averiguar el encoding fuera de pygrep:

```
# En Windows (PowerShell)
[System.IO.File]::ReadAllBytes('archivo.txt') | Select-Object -First 4
# BOM UTF-8: EF BB BF | BOM UTF-16 LE: FF FE | Sin BOM → probablemente latin-1 o cp1252

# En Linux/macOS
file -i archivo.txt

# Con Python
python -c "from charset_normalizer import from_path;
print(from_path('archivo.txt').best())"
```

9. Broken pipe error

Problema: Al usar pipes con `head` o `less`.

Solución: El script ya maneja esto automáticamente. No es un error real, solo una notificación.

```
pygrep "pattern" bigfile.log | head -n 10
# Funciona correctamente, sale con código 0
```

10. ReDoS warning

Problema: El patrón tiene quantifiers anidados.

Solución:

```
# Opción 1: Reescribir el patrón
# En lugar de: (a)+
# Usar: a+

# Opción 2: Si estás seguro, ignorar la advertencia
pygrep "(a)+" archivo.txt

# Opción 3: Si es intencional, usar --redos-strict para validar
pygrep --redos-strict "patrón_seguro" archivo.txt
```

Consejos y Mejores Prácticas

1. Siempre usa comillas en patrones

```
# ☒ Correcto
pygrep "error|warning" archivo.txt
pygrep "[0-9]{4}" archivo.txt

# ☐ Incorrecto (en Windows)
pygrep error|warning archivo.txt
```

2. Usa `-n` para debugging

```
pygrep -n "pattern" archivo.txt
# Te dice exactamente en qué línea está el match
```

3. Combina con `less` para navegación

```
pygrep -C 5 -n "ERROR" bigfile.log | less -R
# -R: interpreta colores ANSI
# En less: /pattern para buscar, n/N para siguiente/anterior, q para salir
```

4. Guarda patrones complejos en archivo

```
# patrones.txt:
error
warning
critical
fatal

# Uso:
pygrep -f patrones.txt -n app.log
```

5. Usa alias para comandos frecuentes

```
# En .bashrc o .zshrc (Linux/macOS)
alias grepn='pygrep -n'
alias grepr='pygrep -r'
alias grepi='pygrep -i'

# En PowerShell (Windows)
function grepn { pygrep -n $args }
function grepr { pygrep -r $args }
```

6. Para buscar en muchos archivos, usa recursividad

```
# ☒ Mejor
pygrep -r "pattern" src/

# ☐ Peor (puede fallar con muchos archivos)
pygrep "pattern" src/* src/**/*
```

7. Filtra por tipo de archivo

```
# Solo Python
pygrep -r "def " --include="*.py" src/

# Solo JavaScript y TypeScript
pygrep -r "function" --include="*.js,ts" src/
```

8. Usa contexto sabiamente

```
# Mucho contexto = output grande
pygrep -C 10 "ERROR" app.log

# Poco contexto = más rápido de leer
pygrep -C 2 "ERROR" app.log
```

9. Para contar, usa `-c`

```
# ☒ Eficiente
pygrep -c "ERROR" app.log
```

```
# ✖ Ineficiente
pygrep "ERROR" app.log | wc -l
```

10. Combina con otras herramientas

```
# Ordenar y contar
pygrep "ERROR" app.log | sort | uniq -c | sort -rn

# Extraer solo la parte que matchea
pygrep -o "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log

# Filtrar múltiples veces
pygrep "ERROR" app.log | pygrep -v "WARNING" | pygrep "database"
```

Referencia Rápida

Comandos Más Comunes

```
# Búsqueda básica
pygrep "pattern" archivo.txt

# Con número de línea
pygrep -n "pattern" archivo.txt

# Ignorar mayúsculas
pygrep -i "pattern" archivo.txt

# Smart-case (insensible si todo minúsculas, sensible si hay mayúscula)
pygrep -S "pattern" archivo.txt

# Recursivo
pygrep -r "pattern" directorio/

# Con contexto
pygrep -C 3 "pattern" archivo.txt

# Contar
pygrep -c "pattern" archivo.txt

# Múltiples patrones
pygrep "error|warning|fatal" archivo.txt

# Desde archivo
pygrep -f patrones.txt archivo.txt

# Fixed string
pygrep -F ".log" archivo.txt
```

```
# Word boundary
pygrep -w "def" archivo.py

# Encoding específico
pygrep --encoding latin-1 "pattern" archivo.txt

# Encoding desconocido → detección automática
pygrep --auto-encoding "pattern" archivo.txt

# Colores forzados
pygrep --color=always "pattern" archivo.txt | less -R
```

Soporte y Contribuciones

Para reportar bugs o solicitar características:

- Revisa la documentación técnica en [TECHNICAL_DOCS.md](#)
- Verifica que no sea un problema conocido (sección Solución de Problemas)
- Proporciona: versión de Python, sistema operativo, comando exacto, archivo de ejemplo (si aplica)

Licencia

MIT License - Uso libre para cualquier propósito.
